



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Camacho Ignacio Violeta

N° de Cuenta: 319061345

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

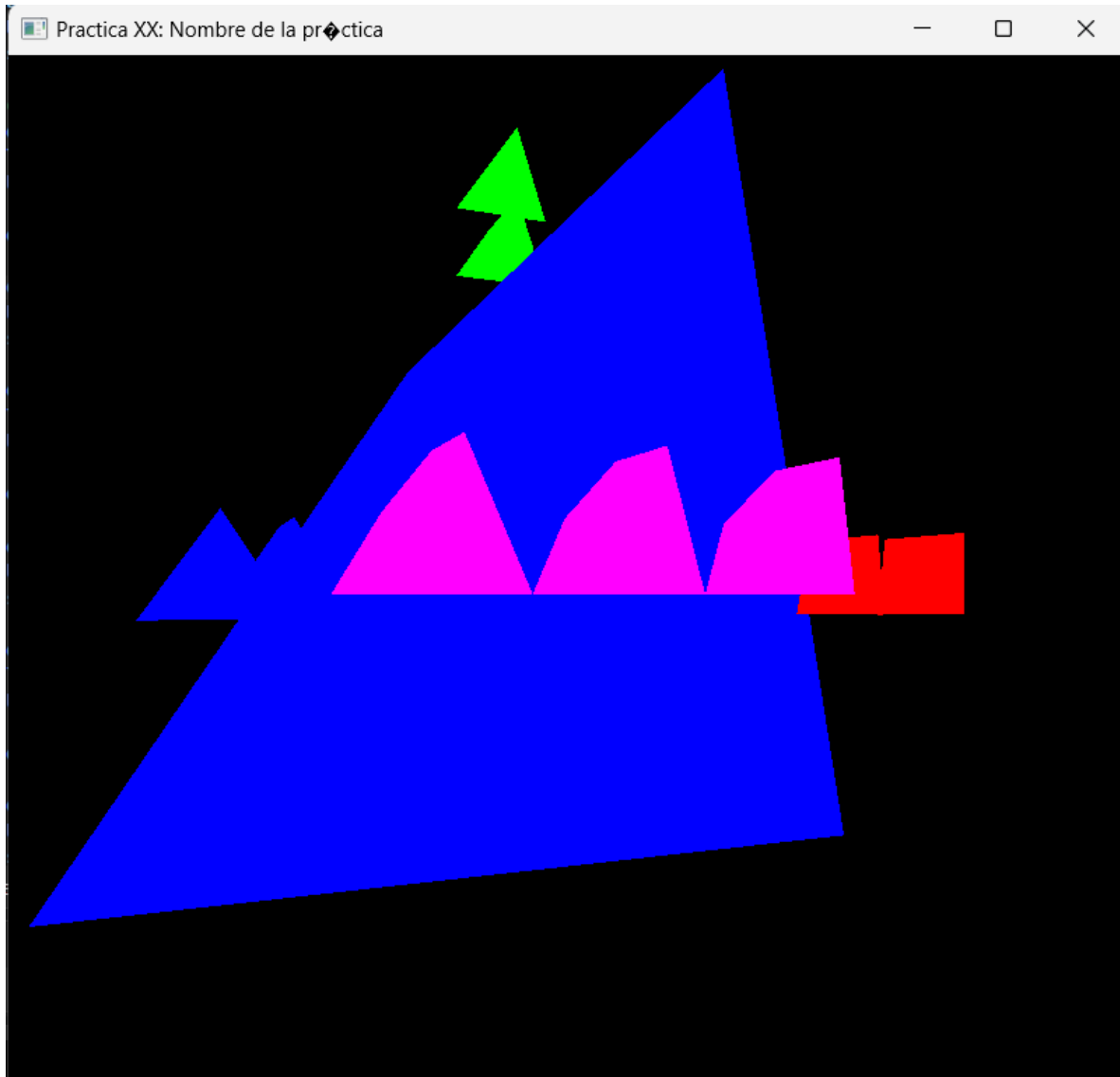
SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 05-03-2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.



El objetivo principal fue implementar modelado geométrico básico (cubo y pirámide triangular regular), aplicar transformaciones (traslación, rotación y escala) y controlar la cámara mediante teclado y mouse.

```

85 // Pirámide triangular regular
86 void CrearPiramideTriangular()
87 {
88     unsigned int indices[] = {
89         0,1,2,
90         0,3,1,
91         0,2,3,
92         1,3,2
93     };
94
95     GLfloat vertices[] = {
96         0.5f, 0.5f, 0.5f, // 0
97         -0.5f, -0.5f, 0.5f, // 1
98         -0.5f, 0.5f, -0.5f, // 2
99         0.5f, -0.5f, -0.5f // 3
100     };
101
102     Mesh* obj1 = new Mesh();
103     obj1->CreateMesh(vertices, indices, 12, 12);
104     meshList.push_back(obj1);
105 }
106

```

Se definieron 4 vértices, se definieron 4 caras triangulares mediante índices y se realizó el ajuste de los vértices para obtener la pirámide regular deseada para el pyraminx, se envió la geometría al objeto Mesh, la pirámide se almacenó en el vector meshList.

```

213 ///EJES x y z
214 model = glm::mat4(1.0f);
215 color = glm::vec3(1.0f, 0.0f, 0.0f);
216 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos q se actualice e shader
217
218 model = glm::translate(model, glm::vec3(6.0f, 0.0f, 0.0f));
219
220 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
221 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
222 meshList[1]->RenderMeshGeometry();
223
224 model = glm::mat4(1.0f);
225 color = glm::vec3(1.0f, 0.0f, 0.0f);
226 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos q se actualice e shader
227
228 model = glm::translate(model, glm::vec3(5.0f, 0.0f, 0.0f));
229
230 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
231 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
232 meshList[1]->RenderMeshGeometry();
233
234 model = glm::mat4(1.0f);
235 color = glm::vec3(0.0f, 1.0f, 0.0f);
236 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos q se actualice e shader
237
238 model = glm::translate(model, glm::vec3(0.0f, 6.0f, 0.0f));

```

```

246 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos q se actualice e shader
247
248 model = glm::translate(model, glm::vec3(0.0f, 5.0f, 0.0f));
249
250 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
251 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
252 meshList[1] -> RenderMeshGeometry();
253
254 model = glm::mat4(1.0f);
255 color = glm::vec3(0.0f, 0.0f, 1.0f);
256 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos q se actualice e shader
257
258 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 6.0f));
259
260 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
261 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
262 meshList[1] -> RenderMeshGeometry();
263
264 model = glm::mat4(1.0f);
265 color = glm::vec3(0.0f, 0.0f, 1.0f);
266 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos q se actualice e shader
267
268 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 5.0f));
269
270 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
271 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
272 meshList[1] -> RenderMeshGeometry();
273

```

Para esta sección se colocaron 2 triangulos de color rojo, verde y azul simplemente para visualizar los ejes X, Y, y Z

```

274 //cara 0
275 model = glm::mat4(1.0f);
276 color = glm::vec3(1.0f, 0.0f, 1.0f);
277 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos q se actualice e shader
278
279 model = glm::translate(model, glm::vec3(4.5f, 0.5f, 1.5f));
280
281 model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
282 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
283 meshList[1] -> RenderMeshGeometry();
284 //
285 model = glm::mat4(1.0f);
286 color = glm::vec3(1.0f, 0.0f, 1.0f);
287 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos q se actualice e shader
288
289 model = glm::translate(model, glm::vec3(3.0f, 0.5f, 3.0f));
290
291 model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
292 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
293 meshList[1] -> RenderMeshGeometry();
294 //
295 model = glm::mat4(1.0f);
296 color = glm::vec3(1.0f, 0.0f, 1.0f);
297 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos q se actualice e shader
298
299 model = glm::translate(model, glm::vec3(1.5f, 0.5f, 4.5f));
300
301 model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
302 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
303 meshList[1] -> RenderMeshGeometry();
304

```

Y para esta ultima parte se encuentran las instancias de 3 triangulos en color morado que iban a utilizarse para hacer el llenado de la cara 0

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

La implementación planeada era relativamente sencilla en concepto: Crear 9 pirámides de cada color e incrustarlas dentro de una pirámide negra más grande.

Sin embargo, el problema surgió al calcular correctamente:

Las posiciones exactas de cada pirámide pequeña.

Las traslaciones necesarias en los tres ejes.

El alineamiento correcto respecto a las caras inclinadas.

específicamente:

Determinar cuánto mover en X, Y y Z.

Evitar que las pirámides quedaran flotando o atravesando la estructura.

Ajustar escala proporcional a la pirámide grande.

Se resolvió parcialmente ya que se logró posicionar algunas pirámides correctamente, pero no se completó la estructura total debido a:

Falta de tiempo.

Carga de otras tareas académicas.

Complejidad geométrica mayor a la estimada inicialmente.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

Aunque el ejercicio parecía sencillo en teoría, la complejidad aumentó debido al manejo de matrices 4x4, transformaciones encadenadas, cálculo espacial tridimensional, comprensión de proyección perspectiva. El modelado básico fue sencillo, pero el posicionamiento estructural fue lo más complejo.

b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

En esta ocasión la explicación fue mucho más clara y sencilla de entender, al igual que la complejidad de el ejercicio de clase me pareció perfectamente acorde a lo revisado en clase, sin embargo con respecto a eso la complejidad de este ejercicio resultó mucho mayor por los puntos mencionados anteriormente, fuera de eso se agradece profundamente que se haya ajustado el ritmo de las explicaciones y demostraciones según la retroalimentación dada.

c. Conclusión

La práctica permitió comprender cómo funciona el pipeline gráfico, cómo se envían matrices al shader, cómo funciona una cámara sintética, así como la importancia del orden de transformaciones.

Aunque no se completó el diseño final planeado, se logró entender la lógica fundamental del modelado geométrico en OpenGL, fue una práctica retadora pero formativa.

4. Bibliografía en formato APA

GLFW. (2023). GLFW Documentation. <https://www.glfw.org/docs/latest/>

OpenGL. (2023). OpenGL Reference Pages. <https://www.khronos.org/opengl/>

GLM. (2023). OpenGL Mathematics (GLM) Manual. <https://glm.g-truc.net/>